# Desain Lapisan Processing (Hive dan Spark SQL): Pemantauan Titik Api Indonesia

Teknologi Big Data - Topic 10 - Tugas Kelompok

| Kelompok 6 | NPM | Pembagian tugas |
|---|---|---|
| Muhammad Bryan Farras | 2306230975 | Bab 1 (konteks kasus, peran layer processing, 3 query) & Bab 4 (analisis waktu, format, partisi, 100x) |
| Mirza Adi Raffiansyah | 2306210323 | Bab 2 (desain skema tabel Teks, Parquet, Partisi tanggal, & tabel referensi provinsi) |
| Calvin Wirathama Katoroy | 2306242395 | Bab 3 (instalasi Docker, Hive 4.2.1, PySpark 4.2.0, hands-on, screenshot, uji waktu median 3x) |
| Salim | 2306204604 | Bab 5 (satu halaman Catatan Keputusan Desain), Diagram Arsitektur v0.2, & penyusunan PDF |

## Bab 1. Konteks Kasus Kelompok dan Peran Lapisan Processing

**Keterkaitan dengan kasus kelompok.** Laporan ini melanjutkan arsitektur sistem **pemantauan titik api (*hotspot*) kebakaran hutan dan lahan di Indonesia** menggunakan data satelit **VIIRS NASA FIRMS** (*sea_fires_7d.csv*, 19–25 September 2026) yang telah dirancang lapisan *storage*-nya pada **Topic 9 (Apache HBase)**. Sampel wilayah Indonesia berisi **54.516 baris** (rata-rata 7.788 titik/hari, puncak 10.974 titik pada 19 Sep dan 5.686 titik pada 25 Sep) yang tersebar pada **300 sel grid** (1° x 1°). Pada Topic 9, tabel HBase `hotspot` (`grid#tanggal_terbalik#jam#urut`) menjawab *get* satu titik (0,073 detik) dan *range scan* satu sel (0,967 detik untuk 925 baris `g087112`), tetapi HBase tidak mendukung `JOIN` maupun agregasi `GROUP BY`. Oleh sebab itu, **Pertanyaan 4** (penyaringan lintas sel tiap jam untuk peringatan dini) dan **Pertanyaan 5** (join ke batas provinsi & agregasi harian) dikerjakan di **lapisan processing** menggunakan **Apache Hive** dan **Spark SQL**.

| No | Pertanyaan Data Kasus Titik Api | Pola Akses | Frekuensi | Lapisan & Mesin Penjawab |
|---|---|---|---|---|
| 1 | Tampilkan 20 titik api terbaru pada sel grid X (peta wilayah) | Scan rentang sempit | Setiap peta dibuka | Storage Operasional: **HBase** (Topic 9) |
| 2 | Tampilkan detail satu titik api saat diklik pada peta | Lookup per key (get) | Puluhan kali/menit | Storage Operasional: **HBase** (Topic 9) |
| 3 | Titik api pada sel X dalam N hari terakhir | Scan rentang sempit | Beberapa kali/menit | Storage Operasional: **HBase** (Topic 9) |
| 4 | Peringatan dini titik api berbahaya (FRP >= 10 MW / high) pada hari puncak | Filter partisi & agregasi | Tiap jam | Processing: **Spark SQL / Hive (Query 3)** |
| 5 | Jumlah titik api dan intensitas FRP per kategori serta per provinsi | Agregasi & Join referensi | Tiap jam / harian | Processing: **Spark SQL / Hive (Query 1 & 2)** |

## Bab 2. Desain Skema Tabel (Teks, Parquet, Berpartisi) dan Tiga Query Uji

**Ukuran data uji (1x dan 10x).** Pengujian dilakukan pada dua skala data di direktori `/data` (`/root/topic10-data` pada filesystem `ext4` WSL2): **(1) Data Kecil (**`1x` **= 54.516 baris, 4,71 MB CSV)** berisi seluruh titik api Indonesia selama 7 hari (19–25 Sep 2026) dengan statistik identik Topic 9 (*nominal* 50.095, *low* 2.331, *high* 2.090; sel terpadat `g087112` = 5.107 baris); **(2) Data Diperbesar (**`10x` **= 545.160 baris, 47,71 MB CSV)** sebagai replikasi 10x lipat untuk menguji skalabilitas di luar waktu *start-up*; serta **(3) Tabel Referensi** `ref_grid_provinsi` **(300 baris)** yang memetakan seluruh 300 kode `grid_id` ke `provinsi`, `pulau`, `tingkat_rawan`, dan `luas_gambut_ha`.

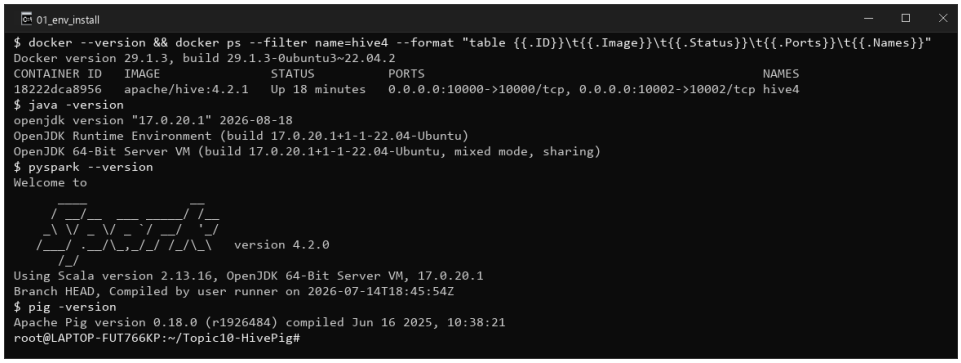
| Varian Tabel | Nama Tabel (1x / 10x) | Definisi Penyimpanan (Hive & Spark SQL) | Perilaku Saat Query Berjalan |
|---|---|---|---|
| **1. Teks (CSV)** | hotspot_1x_text / hotspot_10x_text | EXTERNAL TABLE ... ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' STORED AS TEXTFILE | Membaca seluruh baris dan seluruh 14 kolom dari disk serta mengurai (parse) teks ke angka di CPU. |
| **2. Parquet** | hotspot_1x_parquet / hotspot_10x_parquet | STORED AS PARQUET (kolumnar biner terkompresi, tanpa partisi) | Hanya membaca kolom yang diminta query (column projection) + memanfaatkan statistik min/max per blok. |
| **3. Parquet Partisi** | hotspot_1x_part / hotspot_10x_part | PARTITIONED BY (acq_date STRING) STORED AS PARQUET (7 partisi: 2026-09-19 s/d 2026-09-25) | Proyeksi kolom Parquet + Partition Pruning: WHERE acq_date hanya membuka folder tanggal tersebut. |

**Alasan pemilihan kolom partisi** `acq_date`**.** Kolom `acq_date` memiliki kardinalitas terukur (7 partisi/minggu, +-365 partisi/tahun) dan selalu menjadi filter utama laporan harian. Kami tidak mempartisi berdasar `hotspot_id` (54.516 nilai unik) karena akan menciptakan puluhan ribu folder kecil (*small files problem*) yang membebani Metastore dan HDFS.

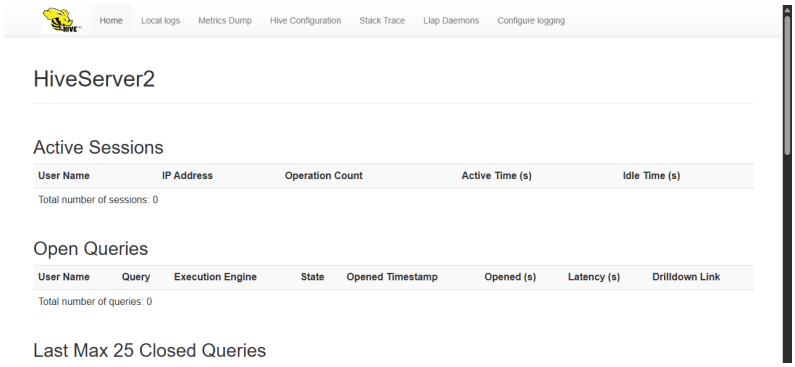
| Query | Tujuan Analitik pada Kasus Titik Api | Potongan Inti Query SQL (Sama di Hive dan Spark SQL) |
|---|---|---|
| **Q1_Agregasi** | Menghitung jumlah titik api, total FRP, rata-rata FRP, dan maksimum FRP per tingkat confidence & siang/malam. | `SELECT confidence, daynight, COUNT(*) AS jumlah_hotspot, ROUND(SUM(frp),2) AS total_frp_mw, ROUND(AVG(frp),2) AS rata_frp_mw, ROUND(MAX(frp),2) AS max_frp_mw FROM <tabel> GROUP BY confidence, daynight ORDER BY total_frp_mw DESC;` |
| **Q2_Join** | Menggabungkan titik api (nominal & high) dengan tabel referensi 300 sel grid untuk Top 10 Provinsi Terdampak. | `SELECT p.provinsi, p.pulau, COUNT(h.hotspot_id) AS jumlah_hotspot, ROUND(SUM(h.frp),2) AS total_frp_mw, ROUND(AVG(h.frp),2) AS rata_frp_mw FROM <tabel> h JOIN ref_grid_provinsi p ON h.grid_id = p.grid_id WHERE h.confidence IN ('nominal','high') GROUP BY p.provinsi, p.pulau ORDER BY jumlah_hotspot DESC LIMIT 10;` |
| **Q3_Partisi** | Menyaring titik api berbahaya (FRP >= 10 MW) pada tanggal puncak kemarau (2026-09-19) per sel grid. | `SELECT grid_id, confidence, COUNT(*) AS jumlah_hotspot, ROUND(SUM(frp),2) AS total_frp_mw, ROUND(MAX(frp),2) AS max_frp_mw FROM <tabel> WHERE acq_date = '2026-09-19' AND frp >= 10.0 GROUP BY grid_id, confidence ORDER BY total_frp_mw DESC LIMIT 10;` |

## Bab 3. Bukti Hands-on Instalasi, Pembuatan Tabel, dan Hasil Query

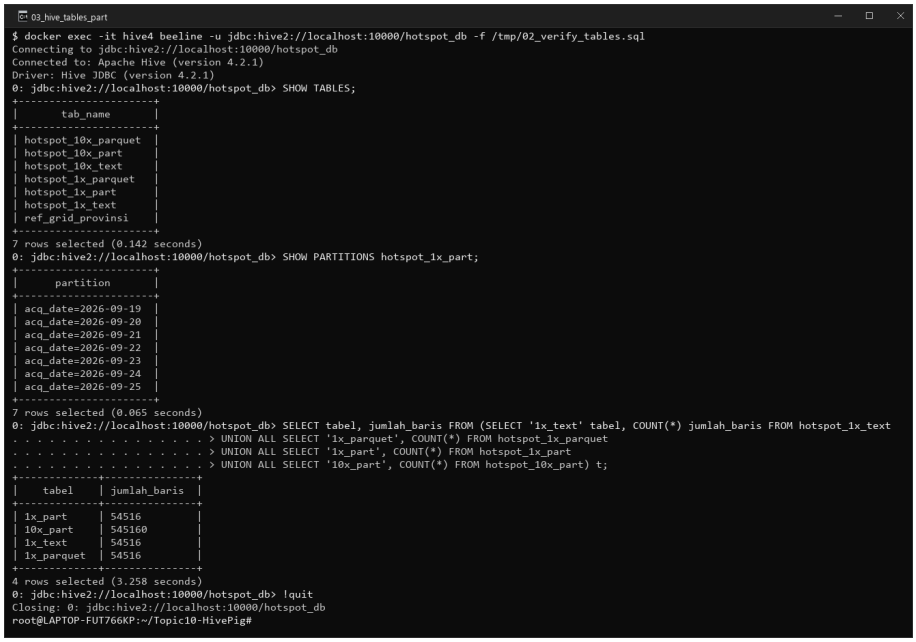
**Lingkungan pengujian.** Seluruh pengukuran dijalankan pada mesin yang sama: **Windows 11 + WSL2 (Ubuntu 22.04 LTS)**, prosesor **Intel Core Ultra 7 155H (22 vCPU)**, **RAM 7,5 GiB**, SSD NVMe (`ext4`). **Apache Hive 4.2.1** dijalankan lewat kontainer resmi `apache/hive:4.2.1` di **Docker 29.1.3** (membawa JDK 21, HiveServer2, Derby Metastore, dan engine **Apache Tez**). **Apache Spark SQL** dijalankan lewat **PySpark 4.2.0** (`pip install pyspark==4.2.0 pyarrow`) di atas **OpenJDK 17.0.20.1** dan Python 3.10.12 dengan mode `local[*]` (22 thread). Sebagai pelengkap opsional materi kuliah, **Apache Pig 0.18.0** juga telah dipasang dan diverifikasi.




**Gambar 1.** Verifikasi instalasi Docker 29.1.3, kontainer hive4 (apache/hive:4.2.1 aktif di port 10000 & 10002), OpenJDK 17.0.20.1, PySpark 4.2.0, dan Pig 0.18.0. Menjawab: lingkungan kerja berjalan.

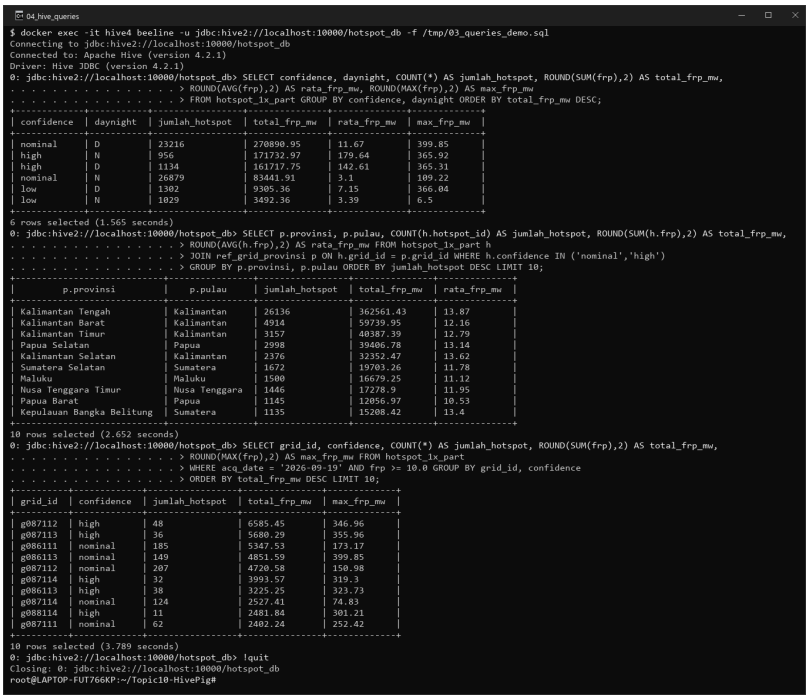



**Gambar 2.** Antarmuka web HiveServer2 (http://localhost:10002) menampilkan layanan Apache Hive 4.2.1 aktif beserta riwayat eksekusi query Tez. Menjawab: server Hive & Metastore hidup.



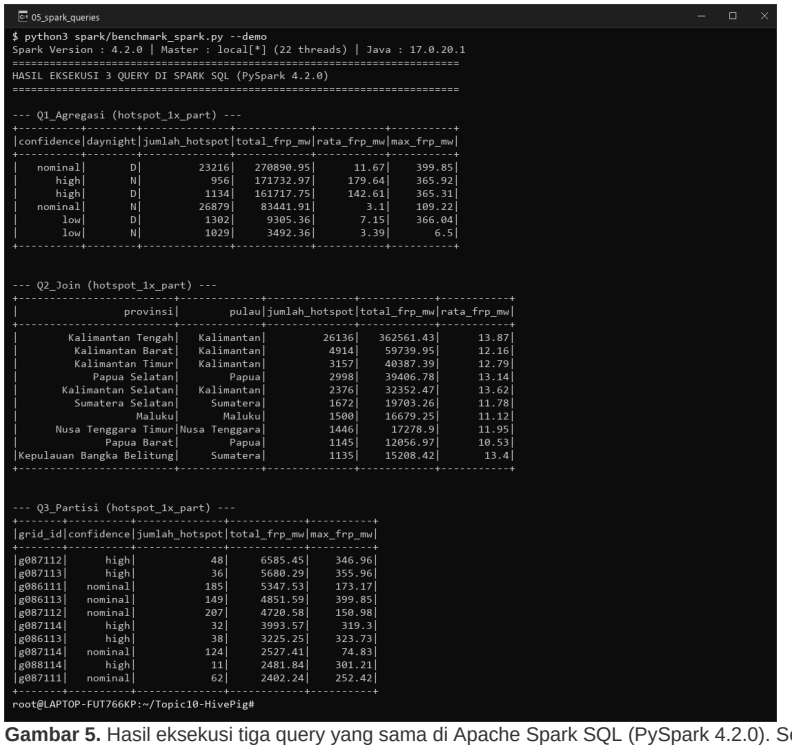

**Gambar 3.** Beeline Hive: SHOW TABLES (7 tabel), SHOW PARTITIONS hotspot_1x_part (7 partisi tanggal), dan verifikasi jumlah baris (54.516 pada 1x dan 545.160 pada 10x). Menjawab: tabel Teks, Parquet, dan Partisi terbentuk.

## Bab 3 (Lanjutan). Bukti Hasil Eksekusi Tiga Query dan Pengukuran Waktu



```
04_hive_queries
$ docker exec -it hive4 beeline -u jdbc:hive2://localhost:10000/hotspot_db -f /tmp/03_queries_demo.sql
Connecting to jdbc:hive2://localhost:10000/hotspot_db
Connected to: Apache Hive (version 4.2.1)
Driver: Hive JDBC (version 4.2.1)
0: jdbc:hive2://localhost:10000/hotspot_db> SELECT confidence, daynight, COUNT(*) AS jumlah_hotspot, ROUND(SUM(frp),2) AS total_frp_mw,
. . . . . . . . . . . . . . . . . . . . . > ROUND(AVG(frp),2) AS rata_frp_mw, ROUND(MAX(frp),2) AS max_frp_mw
. . . . . . . . . . . . . . . . . . . . . > FROM hotspot_1x_part GROUP BY confidence, daynight ORDER BY total_frp_mw DESC;
+------------+----------+---------------+--------------+-------------+-------------+
| confidence | daynight | jumlah_hotspot | total_frp_mw | rata_frp_mw | max_frp_mw |
+------------+----------+---------------+--------------+-------------+-------------+
| nominal    | D        | 23216          | 270890.95    | 11.67       | 399.85      |
| high       | N        | 956            | 171732.97    | 179.64      | 365.92      |
| high       | D        | 1134           | 161717.75    | 142.61      | 365.31      |
| nominal    | N        | 26879          | 83441.91     | 3.1         | 109.22      |
| low        | D        | 1302           | 9305.36      | 7.15        | 366.04      |
| low        | N        | 1029           | 3492.36      | 3.39        | 6.5         |
+------------+----------+---------------+--------------+-------------+-------------+
6 rows selected (1.565 seconds)
0: jdbc:hive2://localhost:10000/hotspot_db> SELECT p.provinsi, p.pulau, COUNT(h.hotspot_id) AS jumlah_hotspot, ROUND(SUM(h.frp),2) AS total_frp_mw,
. . . . . . . . . . . . . . . . . . . . . > ROUND(AVG(h.frp),2) AS rata_frp_mw FROM hotspot_1x_part h
. . . . . . . . . . . . . . . . . . . . . > JOIN ref_grid_provinsi p ON h.grid_id = p.grid_id WHERE h.confidence IN ('nominal','high')
. . . . . . . . . . . . . . . . . . . . . > GROUP BY p.provinsi, p.pulau ORDER BY jumlah_hotspot DESC LIMIT 10;
+--------------------------+-------------+---------------+--------------+-------------+
|        p.provinsi        |    p.pulau  | jumlah_hotspot | total_frp_mw | rata_frp_mw |
+--------------------------+-------------+---------------+--------------+-------------+
| Kalimantan Tengah        | Kalimantan  | 26136          | 362561.43    | 13.87       |
| Kalimantan Barat         | Kalimantan  | 4914           | 59739.95     | 12.16       |
| Kalimantan Timur         | Kalimantan  | 3157           | 40387.39     | 12.79       |
| Papua Selatan            | Papua       | 2998           | 39406.78     | 13.14       |
| Kalimantan Selatan       | Kalimantan  | 2376           | 32352.47     | 13.62       |
| Sumatera Selatan         | Sumatera    | 1672           | 19703.26     | 11.78       |
| Maluku                   | Maluku      | 1500           | 16679.25     | 11.12       |
| Nusa Tenggara Timur      | Nusa Tenggara | 1446         | 17278.9      | 11.95       |
| Papua Barat              | Papua       | 1145           | 12056.97     | 10.53       |
| Kepulauan Bangka Belitung | Sumatera   | 1135           | 15208.42     | 13.4        |
+--------------------------+-------------+---------------+--------------+-------------+
10 rows selected (2.652 seconds)
0: jdbc:hive2://localhost:10000/hotspot_db> SELECT grid_id, confidence, COUNT(*) AS jumlah_hotspot, ROUND(SUM(frp),2) AS total_frp_mw,
. . . . . . . . . . . . . . . . . . . . . > ROUND(MAX(frp),2) AS max_frp_mw FROM hotspot_1x_part
. . . . . . . . . . . . . . . . . . . . . > WHERE acq_date = '2026-09-19' AND frp >= 10.0 GROUP BY grid_id, confidence
. . . . . . . . . . . . . . . . . . . . . > ORDER BY total_frp_mw DESC LIMIT 10;
+---------+------------+---------------+--------------+-------------+
| grid_id | confidence | jumlah_hotspot | total_frp_mw | max_frp_mw |
+---------+------------+---------------+--------------+-------------+
| g087112 | high       | 48             | 6585.45      | 346.96      |
| g087113 | high       | 36             | 5680.29      | 355.96      |
| g086111 | nominal    | 185            | 5347.53      | 173.17      |
| g086113 | nominal    | 149            | 4851.59      | 399.85      |
| g087112 | nominal    | 207            | 4720.58      | 150.98      |
| g087114 | high       | 32             | 3993.57      | 319.3       |
| g086113 | high       | 38             | 3225.25      | 323.73      |
| g087114 | nominal    | 124            | 2527.41      | 74.83       |
| g088114 | high       | 11             | 2481.84      | 301.21      |
| g087111 | nominal    | 62             | 2402.24      | 252.42      |
+---------+------------+---------------+--------------+-------------+
10 rows selected (3.789 seconds)
0: jdbc:hive2://localhost:10000/hotspot_db> !quit
Closing: 0: jdbc:hive2://localhost:10000/hotspot_db
root@LAPTOP-FUT766KP:~/Topic10-HivePig#
```

**Gambar 4.** Hasil eksekusi tiga query (Q1_Agregasi, Q2_Join, Q3_Partisi) pada Apache Hive 4.2.1. Kalimantan Tengah tertinggi (26.136 titik, 362.561,43 MW) dan sel g087112 terpanas pada 19 Sep (6.585,45 MW).



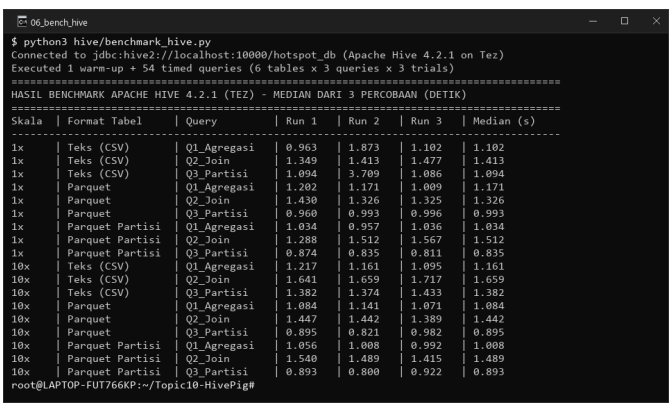
```
05_spark_queries
$ python3 spark/benchmark_spark.py --demo
Spark Version : 4.2.0 | Master : local[*] (22 threads) | Java : 17.0.20.1
======================================================================
HASIL EKSEKUSI 3 QUERY DI SPARK SQL (PySpark 4.2.0)
======================================================================

--- Q1_Agregasi (hotspot_1x_part) ---
+----------+--------+-------------+------------+-----------+----------+
|confidence|daynight|jumlah_hotspot|total_frp_mw|rata_frp_mw|max_frp_mw|
+----------+--------+-------------+------------+-----------+----------+
|   nominal|       D|         23216|   270890.95|      11.67|    399.85|
|      high|       N|           956|   171732.97|     179.64|    365.92|
|      high|       D|          1134|   161717.75|     142.61|    365.31|
|   nominal|       N|         26879|    83441.91|        3.1|    109.22|
|       low|       D|          1302|     9305.36|       7.15|    366.04|
|       low|       N|          1029|     3492.36|       3.39|       6.5|
+----------+--------+-------------+------------+-----------+----------+


--- Q2_Join (hotspot_1x_part) ---
+--------------------------+-------------+-------------+------------+-----------+
|                 provinsi|        pulau|jumlah_hotspot|total_frp_mw|rata_frp_mw|
+--------------------------+-------------+-------------+------------+-----------+
|        Kalimantan Tengah|   Kalimantan|        26136|   362561.43|      13.87|
|         Kalimantan Barat|   Kalimantan|         4914|    59739.95|      12.16|
|         Kalimantan Timur|   Kalimantan|         3157|    40387.39|      12.79|
|            Papua Selatan|       Papua|         2998|    39406.78|      13.14|
|       Kalimantan Selatan|   Kalimantan|         2376|    32352.47|      13.62|
|         Sumatera Selatan|     Sumatera|         1672|    19703.26|      11.78|
|                  Maluku|       Maluku|         1500|    16679.25|      11.12|
|      Nusa Tenggara Timur|Nusa Tenggara|         1446|     17278.9|      11.95|
|             Papua Barat|       Papua|         1145|    12056.97|      10.53|
|Kepulauan Bangka Belitung|     Sumatera|         1135|    15208.42|       13.4|
+--------------------------+-------------+-------------+------------+-----------+

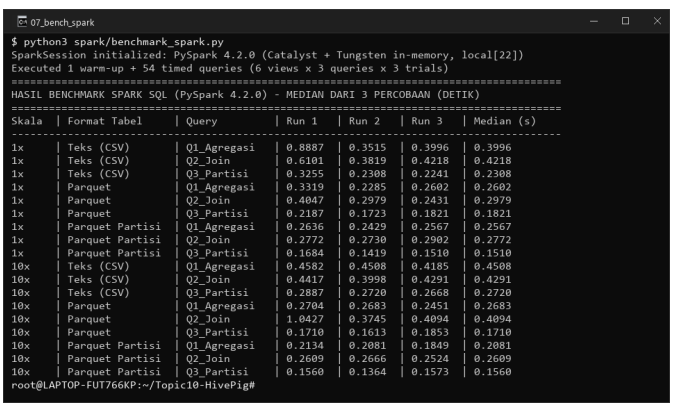

--- Q3_Partisi (hotspot_1x_part) ---
+-------+----------+-------------+------------+----------+
|grid_id|confidence|jumlah_hotspot|total_frp_mw|max_frp_mw|
+-------+----------+-------------+------------+----------+
|g087112|      high|           48|     6585.45|    346.96|
|g087113|      high|           36|     5680.29|    355.96|
|g086111|   nominal|          185|     5347.53|    173.17|
|g086113|   nominal|          149|     4851.59|    399.85|
|g087112|   nominal|          207|     4720.58|    150.98|
|g087114|      high|           32|     3993.57|     319.3|
|g086113|      high|           38|     3225.25|    323.73|
|g087114|   nominal|          124|     2527.41|     74.83|
|g088114|      high|           11|     2481.84|    301.21|
|g087111|   nominal|           62|     2402.24|    252.42|
+-------+----------+-------------+------------+----------+
root@LAPTOP-FUT766KP:~/Topic10-HivePig#
```

**Gambar 5.** Hasil eksekusi tiga query yang sama di Apache Spark SQL (PySpark 4.2.0). Seluruh angka keluaran identik 100% dengan hasil Hive pada Gambar 4. Menjawab: konsistensi hasil lintas mesin.



```
06_bench_hive
$ python3 hive/benchmark_hive.py
Connected to jdbc:hive2://localhost:10000/hotspot_db (Apache Hive 4.2.1 on Tez)
Executed 1 warm-up + 54 timed queries (6 tables x 3 queries x 3 trials)
================================================================================
HASIL BENCHMARK APACHE HIVE 4.2.1 (TEZ) - MEDIAN DARI 3 PERCOBAAN (DETIK)
================================================================================
Skala | Format Tabel      | Query          | Run 1  | Run 2  | Run 3  | Median (s)
--------------------------------------------------------------------------------
1x    | Teks (CSV)        | Q1_Agregasi    | 0.963  | 1.873  | 1.102  | 1.102
1x    | Teks (CSV)        | Q2_Join        | 1.349  | 1.413  | 1.477  | 1.413
1x    | Teks (CSV)        | Q3_Partisi     | 1.094  | 3.709  | 1.086  | 1.094
1x    | Parquet           | Q1_Agregasi    | 1.202  | 1.171  | 1.009  | 1.171
1x    | Parquet           | Q2_Join        | 1.430  | 1.326  | 1.325  | 1.326
1x    | Parquet           | Q3_Partisi     | 0.960  | 0.993  | 0.996  | 0.993
1x    | Parquet Partisi   | Q1_Agregasi    | 1.034  | 0.957  | 1.036  | 1.034
1x    | Parquet Partisi   | Q2_Join        | 1.288  | 1.512  | 1.567  | 1.512
1x    | Parquet Partisi   | Q3_Partisi     | 0.874  | 0.835  | 0.811  | 0.835
10x   | Teks (CSV)        | Q1_Agregasi    | 1.217  | 1.161  | 1.095  | 1.161
10x   | Teks (CSV)        | Q2_Join        | 1.641  | 1.659  | 1.717  | 1.659
10x   | Teks (CSV)        | Q3_Partisi     | 1.382  | 1.374  | 1.433  | 1.382
10x   | Parquet           | Q1_Agregasi    | 1.084  | 1.141  | 1.071  | 1.084
10x   | Parquet           | Q2_Join        | 1.447  | 1.442  | 1.389  | 1.442
10x   | Parquet           | Q3_Partisi     | 0.895  | 0.821  | 0.982  | 0.895
10x   | Parquet Partisi   | Q1_Agregasi    | 1.056  | 1.008  | 0.992  | 1.008
10x   | Parquet Partisi   | Q2_Join        | 1.540  | 1.489  | 1.415  | 1.489
10x   | Parquet Partisi   | Q3_Partisi     | 0.893  | 0.800  | 0.922  | 0.893
root@LAPTOP-FUT766KP:~/Topic10-HivePig#
```

**Gambar 6.** Log hasil pengukuran waktu 3 percobaan & median pada Apache Hive 4.2.1 (Tez) untuk data 1x & 10x.



```
07_bench_spark
$ python3 spark/benchmark_spark.py
SparkSession initialized: PySpark 4.2.0 (Catalyst + Tungsten in-memory, local[22])
Executed 1 warm-up + 54 timed queries (6 views x 3 queries x 3 trials)
================================================================================
HASIL BENCHMARK SPARK SQL (PySpark 4.2.0) - MEDIAN DARI 3 PERCOBAAN (DETIK)
================================================================================
Skala | Format Tabel      | Query          | Run 1  | Run 2  | Run 3  | Median (s)
--------------------------------------------------------------------------------
1x    | Teks (CSV)        | Q1_Agregasi    | 0.8887 | 0.3515 | 0.3996 | 0.3996
1x    | Teks (CSV)        | Q2_Join        | 0.6101 | 0.3819 | 0.4218 | 0.4218
1x    | Teks (CSV)        | Q3_Partisi     | 0.3255 | 0.2308 | 0.2241 | 0.2308
1x    | Parquet           | Q1_Agregasi    | 0.3319 | 0.2285 | 0.2602 | 0.2602
1x    | Parquet           | Q2_Join        | 0.4047 | 0.2979 | 0.2431 | 0.2979
1x    | Parquet           | Q3_Partisi     | 0.2187 | 0.1723 | 0.1821 | 0.1821
1x    | Parquet Partisi   | Q1_Agregasi    | 0.2636 | 0.2429 | 0.2567 | 0.2567
1x    | Parquet Partisi   | Q2_Join        | 0.2772 | 0.2730 | 0.2902 | 0.2772
1x    | Parquet Partisi   | Q3_Partisi     | 0.1684 | 0.1419 | 0.1510 | 0.1510
10x   | Teks (CSV)        | Q1_Agregasi    | 0.4582 | 0.4508 | 0.4185 | 0.4508
10x   | Teks (CSV)        | Q2_Join        | 0.4417 | 0.3998 | 0.4291 | 0.4291
10x   | Teks (CSV)        | Q3_Partisi     | 0.2887 | 0.2720 | 0.2668 | 0.2720
10x   | Parquet           | Q1_Agregasi    | 0.2704 | 0.2683 | 0.2451 | 0.2683
10x   | Parquet           | Q2_Join        | 1.0427 | 0.3745 | 0.4094 | 0.4094
10x   | Parquet           | Q3_Partisi     | 0.1710 | 0.1613 | 0.1853 | 0.1710
10x   | Parquet Partisi   | Q1_Agregasi    | 0.2134 | 0.2081 | 0.1849 | 0.2081
10x   | Parquet Partisi   | Q2_Join        | 0.2609 | 0.2666 | 0.2524 | 0.2609
10x   | Parquet Partisi   | Q3_Partisi     | 0.1560 | 0.1364 | 0.1573 | 0.1560
root@LAPTOP-FUT766KP:~/Topic10-HivePig#
```

**Gambar 7.** Log hasil pengukuran waktu 3 percobaan & median pada Apache Spark SQL (PySpark 4.2.0) untuk data 1x & 10x.

## Bab 4. Perbandingan Waktu Eksekusi Hive vs Spark SQL dan Analisis Teknis

**Tabel perbandingan waktu median (dari 3 kali percobaan).** Setelah 1 kali *warm-up* sesi, setiap query dijalankan 3 kali pada masing-masing kombinasi format tabel (Teks CSV, Parquet, Parquet Partisi) dan ukuran data (**1x = 54.516 baris** serta **10x = 545.160 baris**), lalu diambil nilai **median**-nya:

| Query | Format Tabel | Hive 1x (Run 1/2/3) | Hive 1x Median | Spark 1x Median | Hive 10x (Run 1/2/3) | Hive 10x Median | Spark 10x Median | Percepatan Spark (10x) |
|---|---|---|---|---|---|---|---|---|
| **Q1_Agregasi** | Teks (CSV) | 0,96 / 1,87 / 1,10 | 1,102 s | 0,400 s | 1,22 / 1,16 / 1,10 | 1,161 s | 0,451 s | 2,57x lebih cepat |
| **Q1_Agregasi** | Parquet | 1,20 / 1,17 / 1,01 | 1,171 s | 0,260 s | 1,08 / 1,14 / 1,07 | 1,084 s | 0,268 s | 4,04x lebih cepat |
| **Q1_Agregasi** | Parquet Partisi | 1,03 / 0,96 / 1,04 | 1,034 s | 0,257 s | 1,06 / 1,01 / 0,99 | 1,008 s | 0,208 s | 4,85x lebih cepat |
| **Q2_Join** | Teks (CSV) | 1,35 / 1,41 / 1,48 | 1,413 s | 0,422 s | 1,64 / 1,66 / 1,72 | 1,659 s | 0,429 s | 3,87x lebih cepat |
| **Q2_Join** | Parquet | 1,43 / 1,33 / 1,33 | 1,326 s | 0,298 s | 1,45 / 1,44 / 1,39 | 1,442 s | 0,409 s | 3,53x lebih cepat |
| **Q2_Join** | Parquet Partisi | 1,29 / 1,51 / 1,57 | 1,512 s | 0,277 s | 1,54 / 1,49 / 1,42 | 1,489 s | 0,261 s | 5,70x lebih cepat |
| **Q3_Partisi** | Teks (CSV) | 1,09 / 3,71 / 1,09 | 1,094 s | 0,231 s | 1,38 / 1,37 / 1,43 | 1,382 s | 0,272 s | 5,08x lebih cepat |
| **Q3_Partisi** | Parquet | 0,96 / 0,99 / 1,00 | 0,993 s | 0,182 s | 0,90 / 0,82 / 0,98 | 0,895 s | 0,171 s | 5,23x lebih cepat |
| **Q3_Partisi** | Parquet Partisi | 0,87 / 0,84 / 0,81 | **0,835 s** | **0,151 s** | 0,89 / 0,80 / 0,92 | **0,893 s** | **0,156 s** | **5,72x lebih cepat** |

### Analisis Teknis Hasil Pengukuran

**1. Mengapa format file (Teks CSV lawan Parquet) sangat berpengaruh?**
Pada tabel **Teks (CSV)**, mesin query wajib membaca setiap baris secara penuh (seluruh 14 kolom, 47,71 MB pada data 10x) lalu memecah delimiter koma dan mengonversi teks menjadi tipe numerik (`DOUBLE`/`INT`) di CPU. Sebaliknya, **Apache Parquet** menyimpan data secara **kolumnar biner** terkompresi (ukuran fisik di disk menyusut menjadi +-8,9 MB pada data 10x, menghemat >80% I/O) dan menanamkan statistik `min/max` di setiap *row group*. Saat menjalankan `Q1_Agregasi` yang hanya menyentuh 3 kolom (`confidence`, `daynight`, `frp`), pembaca Parquet langsung melompati 11 kolom lainnya (*column projection*). Di Spark SQL (10x), peralihan dari CSV ke Parquet mempercepat `Q1_Agregasi` dari **0,451 detik menjadi 0,268 detik (1,68x lebih cepat)**, dan di Hive mempercepat `Q3_Partisi` dari **1,382 detik menjadi 0,895 detik (1,54x lebih cepat)**.

**2. Mengapa partisi (`PARTITIONED BY acq_date`) mempercepat query berfilter tanggal?**
Pada `Q3_Partisi` (`WHERE acq_date = '2026-09-19' AND frp >= 10.0`), tabel tanpa partisi harus memeriksa berkas seluruh 7 hari (545.160 baris pada 10x). Dengan **Parquet Berpartisi**, baik Hive maupun Spark melakukan **Partition Pruning**: mesin hanya membuka folder `acq_date=2026-09-19` (109.740 baris = 20,1% dari total data) dan melewati 6 folder tanggal lainnya tanpa membacanya sama sekali. Hasilnya, `Q3_Partisi` pada Parquet Berpartisi mencapai waktu tercepat di seluruh pengujian: **0,835 s (1x)** dan **0,893 s (10x)** di Hive, serta **0,151 s (1x)** dan **0,156 s (10x)** di Spark SQL. Sebaliknya, pada `Q1_Agregasi` dan `Q2_Join` yang tidak memfilter `acq_date`, seluruh 7 partisi tetap harus dibaca sehingga waktunya serupa dengan Parquet non-partisi.

**3. Mengapa Spark SQL 2,6x–5,7x lebih cepat daripada Hive (Tez), dan apa perbedaan sintaksnya?**
• **Perbedaan Mesin Eksekusi**: Walaupun Hive 4.2.1 sudah memakai **Apache Tez** (menghindari tulis ke HDFS antar-tahap seperti MapReduce lama), setiap query tetap melalui kompilasi rencana di HiveServer2, inisialisasi DAG Tez, dan pembacaan dari kontainer. Sebaliknya, **Spark SQL** mengandalkan **Catalyst Optimizer** dan **Tungsten Whole-Stage Code Generation** yang mengeksekusi tugas langsung di memori (*in-memory*) pada 22 thread paralel, serta secara otomatis melakukan **Broadcast Hash Join** terhadap tabel kecil `ref_grid_provinsi` (300 baris) pada `Q2_Join` (0,261 s di Spark vs 1,489 s di Hive, **5,70x lebih cepat**).
• **Perbedaan Sintaks**: Sintaks SQL (`SELECT ... JOIN ... WHERE ... GROUP BY ... ORDER BY`) **100% identik** di kedua mesin. Perbedaannya hanya pada cara mendaftarkan tabel: Hive memakai DDL deklaratif di Metastore (`CREATE EXTERNAL TABLE ... PARTITIONED BY`), sedangkan PySpark memakai API DataFrame (`spark.read.parquet(...).createOrReplaceTempView(...)`).

**4. Evaluasi Skalabilitas: Mengapa kenaikan 1x ke 10x landai, dan apa yang terjadi bila data diperbesar 100x?**
• **Dari 1x (54,5 rb baris) ke 10x (545,1 rb baris)**: Waktu hanya naik tipis (mis. `Q3_Partisi` CSV di Hive dari 1,094 s ke 1,382 s; Parquet Partisi di Spark dari 0,151 s ke 0,156 s). Ini membuktikan bahwa pada ukuran puluhan megabita, waktu eksekusi masih didominasi oleh **overhead tetap (penjadwalan task & inisialisasi DAG)** sebesar +-0,8 detik di Hive dan +-0,15 detik di Spark.
• **Proyeksi pada 100x (5,45 juta baris, +-477 MB CSV) dan Arsip 10 Tahun (85 juta baris, +-7 GB CSV)**: Pada skala 100x ke atas, biaya baca disk (I/O) dan *shuffle* jaringan/memori akan mendominasi. Tabel **Teks (CSV)** akan melambat drastis secara linear, sementara **Parquet Berpartisi** `acq_date` akan menjaga query peringatan harian tetap sub-detik karena hanya memindai 1 partisi tanggal. Dari sisi mesin: **Spark SQL** tetap jauh lebih cepat selama memori RAM klaster cukup menahan *hash table* dan buffer *shuffle*; namun jika memori terbatas saat melakukan *join* ratusan gigabita, Spark rawan mengalami *spill to disk* atau *OutOfMemoryError* (OOM), sedangkan **Hive di atas Tez** lebih stabil untuk pemrosesan *batch* raksasa dengan memori terbatas.

## Bab 5. Catatan Keputusan Desain dan Diagram Arsitektur v0.2

Halaman ini merangkum **Catatan Keputusan Desain** pemilihan mesin query dan tata letak penyimpanan untuk lapisan *processing* proyek pemantauan titik api Indonesia, dilengkapi **Diagram Arsitektur v0.2** yang menambahkan lapisan *processing* di atas lapisan *storage* Topic 9.

| Bagian | Isi Keputusan Desain Proyek Pemantauan Titik Api Indonesia (Kelompok 6) |
|---|---|
| **Keputusan** | Memilih **Apache Spark SQL (PySpark 4.2.0) di atas format kolumnar Apache Parquet yang dipartisi per tanggal (**`acq_date`**)** sebagai mesin *processing* utama proyek, serta mempertahankan **Hive Metastore (Apache Hive 4.2.1)** di atas berkas Parquet yang sama untuk melayani akses SQL deklaratif/JDBC bagi laporan analitik dan BI. |
| **Konteks** | Proyek memproses deteksi titik api VIIRS Indonesia (**54.516 baris/7 hari**, rata-rata **7.788 titik/hari**, puncak **10.974 titik/hari**, proyeksi 10 tahun **+-85 juta baris** pada **300 sel grid**). Lapisan *storage* operasional **HBase 2.6.7** (Topic 9) sudah menjawab *lookup* satu titik (0,073 s) dan *range scan* satu sel (0,967 s), tetapi sistem membutuhkan lapisan *processing* untuk menjawab: **(1) Pertanyaan 4 (peringatan dini tiap jam)** yang menyaring titik api berbahaya (`FRP >= 10–50 MW` / confidence `high`) lintas sel, dan **(2) Pertanyaan 5 (laporan harian per provinsi)** yang menggabungkan (`JOIN`) titik api dengan 300 sel grid batas provinsi (`ref_grid_provinsi`). Selain itu, proyek akan berlanjut ke *streaming* (Topic 11: Kafka) dan pemodelan prediksi berbasis *machine learning*. |
| **Alternatif ditolak** | 1. **Format Teks (CSV) tanpa partisi**: Ditolak karena selalu membaca seluruh 14 kolom dan memindai semua tanggal; pada data 10x, `Q3_Partisi` di CSV (1,382 s di Hive; 0,272 s di Spark) lebih lambat **54%–74%** dibanding Parquet Berpartisi serta memakai ruang disk 5x lebih boros.<br>2. **Apache Hive (Tez) sebagai satu-satunya mesin processing**: Ditolak untuk peringatan dini per jam dan pipeline terpadu karena latensinya pada seluruh pengujian (0,835–1,659 s) **2,57x hingga 5,72x lebih lambat** daripada Spark SQL (0,151–0,451 s), serta tidak mendukung *stream processing* dan ML dalam satu mesin.<br>3. **Apache Pig (Pig Latin)**: Ditolak karena menggunakan bahasa *dataflow* prosedural khusus (diuji pada `hotspot_etl.pig`: 5,48 detik), tidak interaktif, dan tim sudah fasih memakai SQL. |
| **Konsekuensi** | 1. **Kebutuhan Memori (RAM) Lebih Tinggi**: Spark SQL memproses *shuffle* dan `BroadcastHashJoin` di memori, sehingga alokasi RAM executor dan jumlah partisi shuffle harus dijaga agar tidak mengalami OOM saat data mencapai puluhan juta baris.<br>2. **Pipeline Konversi ke Parquet Berpartisi**: Data mentah harus ditulis ke format Parquet berpartisi `acq_date` dengan penggabungan file (`coalesce`/`repartition`) agar tidak menimbulkan ribuan *small files*.<br>3. **Arsitektur Komplementer (HBase + Spark SQL/Hive)**: HBase melayani peta operasional berlatensi milidetik per key, sedangkan Spark SQL & Hive mengolah tabel analitik Parquet berpartisi untuk join provinsi dan agregasi. |
| **Bukti** | Pengukuran median 3 kali percobaan pada data 1x (54.516 baris) dan 10x (545.160 baris) di mesin 22 vCPU / 7,5 GiB RAM (Gambar 3–7):<br>• **Spark SQL vs Hive (10x Parquet Partisi)**: `Q1_Agregasi` **0,208 s vs 1,008 s (4,85x)**; `Q2_Join` **0,261 s vs 1,489 s (5,70x)**; `Q3_Partisi` **0,156 s vs 0,893 s (5,72x)**.<br>• **Parquet Partisi vs Teks CSV (**`Q3_Partisi` **10x)**: Waktu turun dari **1,382 s ke 0,893 s** di Hive (hemat 35,4%) dan dari **0,272 s ke 0,156 s** di Spark SQL (hemat 42,6%) berkat *Partition Pruning* (hanya membaca 1 dari 7 partisi = 109.740 baris). |

**Diagram Arsitektur v0.2 (Storage HBase & Parquet + Layer Processing Spark SQL & Hive)**

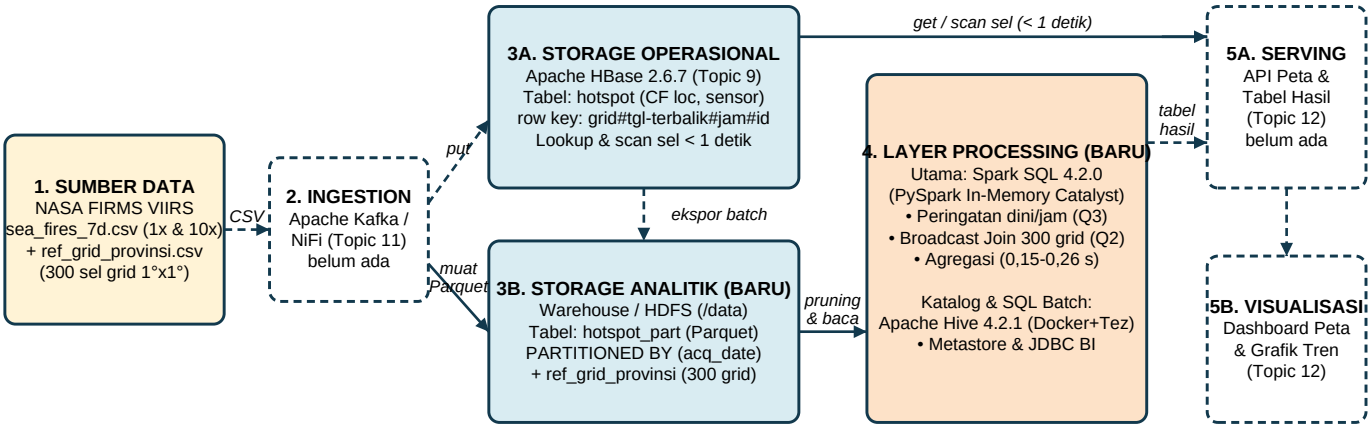



*Diagram Arsitektur v0.2: Garis penuh = sudah dibangun & diuji (Topic 9 Storage HBase + Topic 10 Processing Spark SQL & Hive). Garis putus-putus = Topic 11-12.*